

JOIN

Informatie ophalen uit meerdere tabellen

Carthesisch product

- Combinatie van alle regels uit de ene tabel met alle regels uit de andere tabel
 - Komt overeen met een CROSS JOIN
- Zelden wenselijk, daarom: JOIN

```
SELECT *  
FROM studenten, cijfers;
```

```
SELECT *  
FROM studenten  
CROSS JOIN cijfers;
```

Cartesisch product

Studenten

Studnr	Studnaam
S1	Jan
S2	An

Cijfers

Studnr	Vaknr	Punten
S1	V1	15
S1	V2	8
S2	V2	12

```
SELECT *  
FROM studenten, cijfers;
```

Studnr	Studnaam	Studnr	Vaknr	Punten
S1	Jan	S1	V1	15
S1	Jan	S1	V2	8
S1	Jan	S2	V2	12
S2	An	S1	V1	15
S2	An	S1	V2	8
S2	An	S2	V2	12

JOIN

- Oude notatie: impliciete join (SQL-86/SQL-89)

```
SELECT *  
FROM studenten s, cijfers c  
WHERE s.studnr = c.studnr;
```

- Nieuwe notatie (vanaf SQL-92)

```
SELECT *  
FROM studenten s  
JOIN cijfers c ON s.studnr = c.studnr;
```

- Zowel oude als nieuwe notatie kan gebruikt worden

JOIN

```
SELECT *  
FROM studenten s  
JOIN cijfers c ON s.studnr = c.studnr;
```

Studnr	Studnaam	Studnr	Vaknr	Punten
S1	Jan	S1	V1	15
S1	Jan	S1	V2	8
S1	Jan	S2	V2	12
S2	An	S1	V1	15
S2	An	S1	V2	8
S2	An	S2	V2	12

JOIN

- Geef de naam van alle werknemers die in Brussel werken

```
SELECT wnaam  
FROM werknemers w  
JOIN vestigingen v ON w.vesnaam = v.vesnaam  
WHERE plaats = 'BRUSSEL';
```

JOIN

- Geef voor elke werknemer die vervangen kan worden de plaats waar die werknemer werkt

```
SELECT DISTINCT wnaam, plaats  
FROM werknemers w  
JOIN vestigingen v ON w.vesnaam = v.vesnaam  
JOIN vervangingen ver ON w.wnr = ver.wnr;
```

Autojoin

- Tabel samenvoegen met zichzelf

Autojoin

- Wie verdient meer dan de directeur van de vestiging waar ze werken?

```
SELECT w1.wnaam, w1.fnaam  
FROM werknemers w1  
JOIN werknemers d ON (w1.vesnaam = d.vesnaam)  
WHERE d.fnaam = 'DIRECTEUR'  
AND w1.salaris > d.salaris;
```

Autojoin

- Geef de nummers van werknemers die elkaar kunnen vervangen

```
SELECT ver1.wnr, ver2.wnr  
FROM vervangingen ver1  
JOIN vervangingen ver2 ON ver1.vervanger = ver2.wnr  
WHERE ver1.wnr = ver2.vervanger;
```

- Probleem: dubbele paren (transitiviteit)

Autojoin: transitiviteit

- Uitsluiten van transitiviteit in resultaat tabel

```
SELECT ver1.wnr, ver2.wnr  
FROM vervangingen ver1  
JOIN vervangingen ver2 ON ver1.vervanger = ver2.wnr  
WHERE ver1.wnr = ver2.vervanger  
AND ver1.wnr < ver2.wnr;
```

Autojoin: uitbreiding voorbeeld

- Geef de naam van werknemers die elkaar kunnen vervangen

```
SELECT w1.wnaam, w2.wnaam
FROM vervangingen ver1
JOIN vervangingen ver2 ON ver1.vervanger = ver2.wnr
JOIN werknemers w1 ON w1.wnr = ver1.wnr
JOIN werknemers w2 ON w2.wnr = ver2.wnr
WHERE ver1.wnr = ver2.vervanger
AND ver1.wnr < ver2.wnr;
```

Autojoin: uitbreiding voorbeeld

- Ter info: dezelfde query in SQL1

```
SELECT w1.wnaam, w2.wnaam
FROM vervangingen ver1, vervangingen ver2,
     werknemers w1, werknemers w2
WHERE ver1.vervanger = ver2.wnr
AND ver1.wnr = ver2.vervanger
AND w1.wnr = ver1.wnr
AND w2.wnr = ver2.wnr
AND ver1.wnr < ver2.wnr;
```

Vormen van samenvoegen

- Equijoin (=)
- Bijzonder geval van equijoin is de autojoin
- Alternatieve vormen (nonequijoin)
 - <
 - <=
 - >
 - >=
 - !=
 - BETWEEN

JOIN met <

- Laat voor elke werknemer zien, wie meer dan hen verdient

```
SELECT w1.wnaam, w1.salaris, w2.wnaam, w2.salaris  
FROM werknemers w1  
JOIN werknemers w2 ON w1.salaris < w2.salaris;
```

JOIN met BETWEEN

- The following query lists all employees whose hire date occurred when employee 176 (who is listed in job_history because he changed jobs in 2007) was working at the company

```
SELECT e.employee_id, e.first_name, e.last_name, e.hire_date
FROM   employees e, job_history h
WHERE  h.employee_id = 176
AND    e.hire_date BETWEEN h.start_date AND h.end_date;
```


JOIN met BETWEEN

- Geef namen van werknemers die maximum €100 minder en €100 meer verdienen dan elkaar

```
SELECT w1.wnaam, w2.wnaam
FROM werknemers w1
JOIN werknemers w2
    ON w1.salaris BETWEEN w2.salaris - 100 AND w2.salaris + 100
WHERE w1.wnr < w2.wnr;
```

Extra: JOIN

- JOIN = INNER JOIN
- Andere soorten JOINS mogelijk (zie Data Advanced)
 - LEFT [OUTER] JOIN
 - RIGHT [OUTER] JOIN
 - FULL [OUTER] JOIN

```
SELECT *  
FROM studenten s  
JOIN cijfers c ON s.studnr = c.studnr;
```

```
SELECT *  
FROM studenten s  
INNER JOIN cijfers c ON s.studnr = c.studnr;
```

JOIN met USING

- Als samengevoegde kolommen dezelfde naam hebben

```
SELECT *  
FROM studenten s  
JOIN cijfers c ON s.studnr = c.studnr;
```

```
SELECT *  
FROM studenten  
JOIN cijfers USING (studnr);
```

UNION

- UNION verenigt het resultaat van twee SELECT-opdrachten

alfa	beta
X	1
X	2
Y	1
Y	2

aa	bb
X	1
Z	1
M	3

```
SELECT alfa, beta FROM data  
UNION  
SELECT aa, bb FROM info;
```



alfa	beta
X	1
X	2
Y	1
Y	2
Z	1
M	3

UNION

```
SELECT ...  
FROM ...  
WHERE ...  
UNION  
SELECT ...  
FROM ...  
WHERE ...  
ORDER BY ...
```

UNION

- Het bedrijf wenst een overzicht van alle werknemers met hun nummer, naam, functie en de vermelding of ze te veel of te weinig betaald zijn naargelang hun functie.

```
SELECT wnr, wnaam, 'TE VEEL' betaald
FROM werknemers w
JOIN schalen s ON w.fnaam = s.fnaam
WHERE salaris > maxsal
UNION
SELECT wnr, wnaam, 'TE WEINIG' betaald
FROM werknemers w
JOIN schalen s ON w.fnaam = s.fnaam
WHERE salaris < minsal
ORDER BY betaald DESC;
```

Extra: andere algebraïsche opties

- UNION ($A \cup B$)
 - Unie van twee resultaatsets, geen dubbele resultaten ($\{x \mid (x \in A) \vee (x \in B)\}$)
- UNION ALL ($A \cup B$)
 - Unie van twee resultaatsets, wel dubbele resultaten
- INTERSECT ($A \cap B$)
 - Doorsnede van twee resultaatsets ($\{x \mid (x \in A) \wedge (x \in B)\}$)
 - Kan meestal beter opgelost worden met een INNER JOIN
- MINUS ($A \setminus B$)
 - Verschil van twee resultaatsets ($\{x \mid (x \in A) \wedge (x \notin B)\}$)
 - Kan meestal beter opgelost worden met een LEFT of RIGHT OUTER JOIN